Cosmos EVM 심층 분석: EVM 트랜잭션 처리와 go-ethereum 비교

1. 들어가며

Cosmos EVM(구 Ethermint/Evmos)은 Cosmos SDK 위에서 Ethereum Virtual Machine을 구동하는 프레임워크입니다. 단순히 "EVM을 포팅했다"는 설명 이면에는 두 개의 근본적으로 다른 블록체인 아키텍처를 하나로 결합하는 복잡한 엔지니어링이 숨어 있습니다.

> "Instead of using a trie and database for querying and persistence, Cosmos EVM uses the Cosmos KVStore (key-value store) and Cosmos SDK Keeper to facilitate state transitions."
> — Cosmos EVM Documentation

이 글에서는 다음 세 가지 핵심 질문에 답합니다:

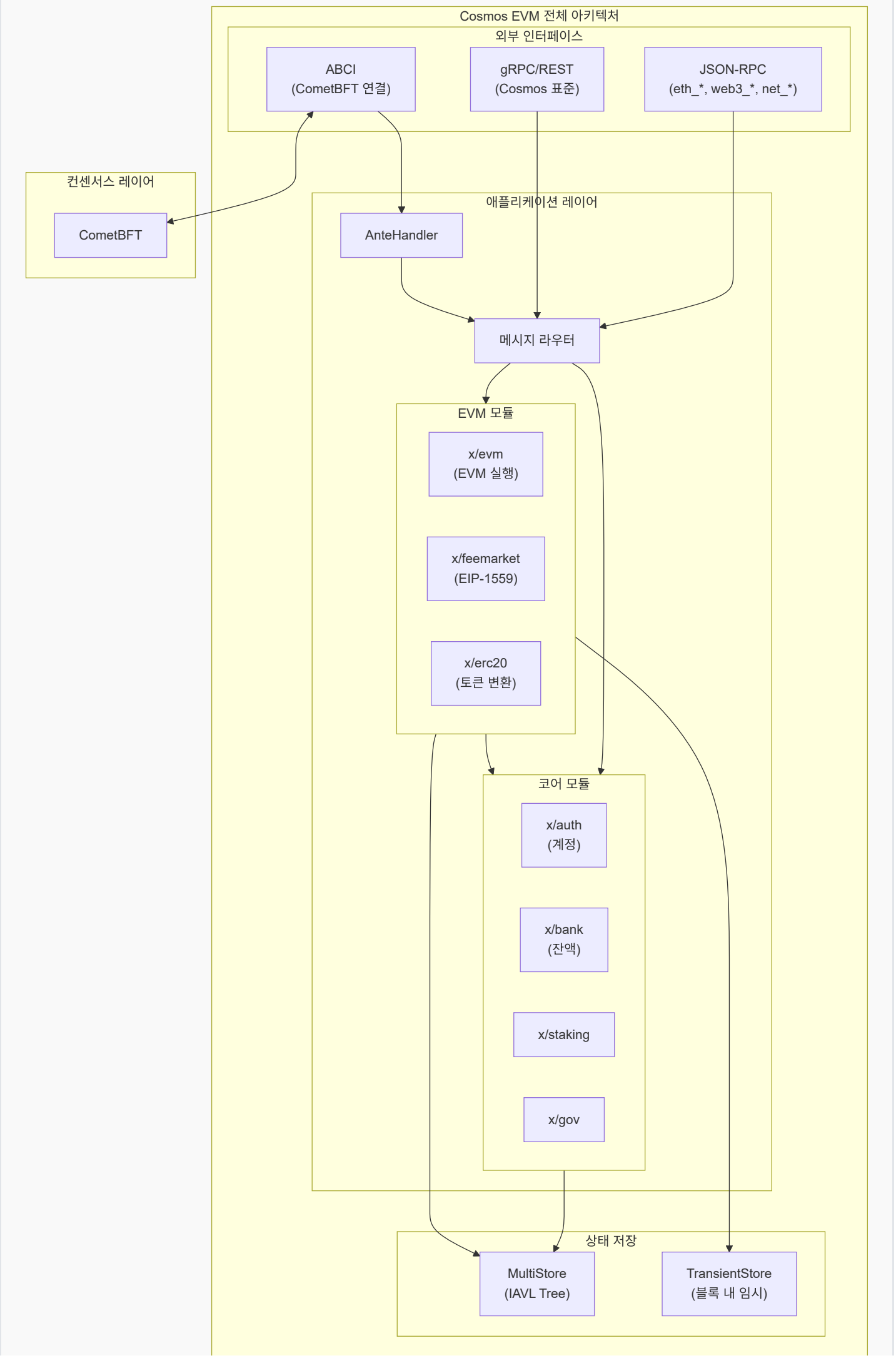
1. **Cosmos SDK와 x/evm 모듈은 어떻게 상호작용하는가?**
2. **EVM 트랜잭션이 Cosmos EVM에서 어떤 경로로 처리되는가?**
3. **go-ethereum과 Cosmos EVM의 StateDB, Precompile 구현은 어떻게 다른가?**

---

2. Cosmos SDK와 Cosmos EVM의 아키텍처

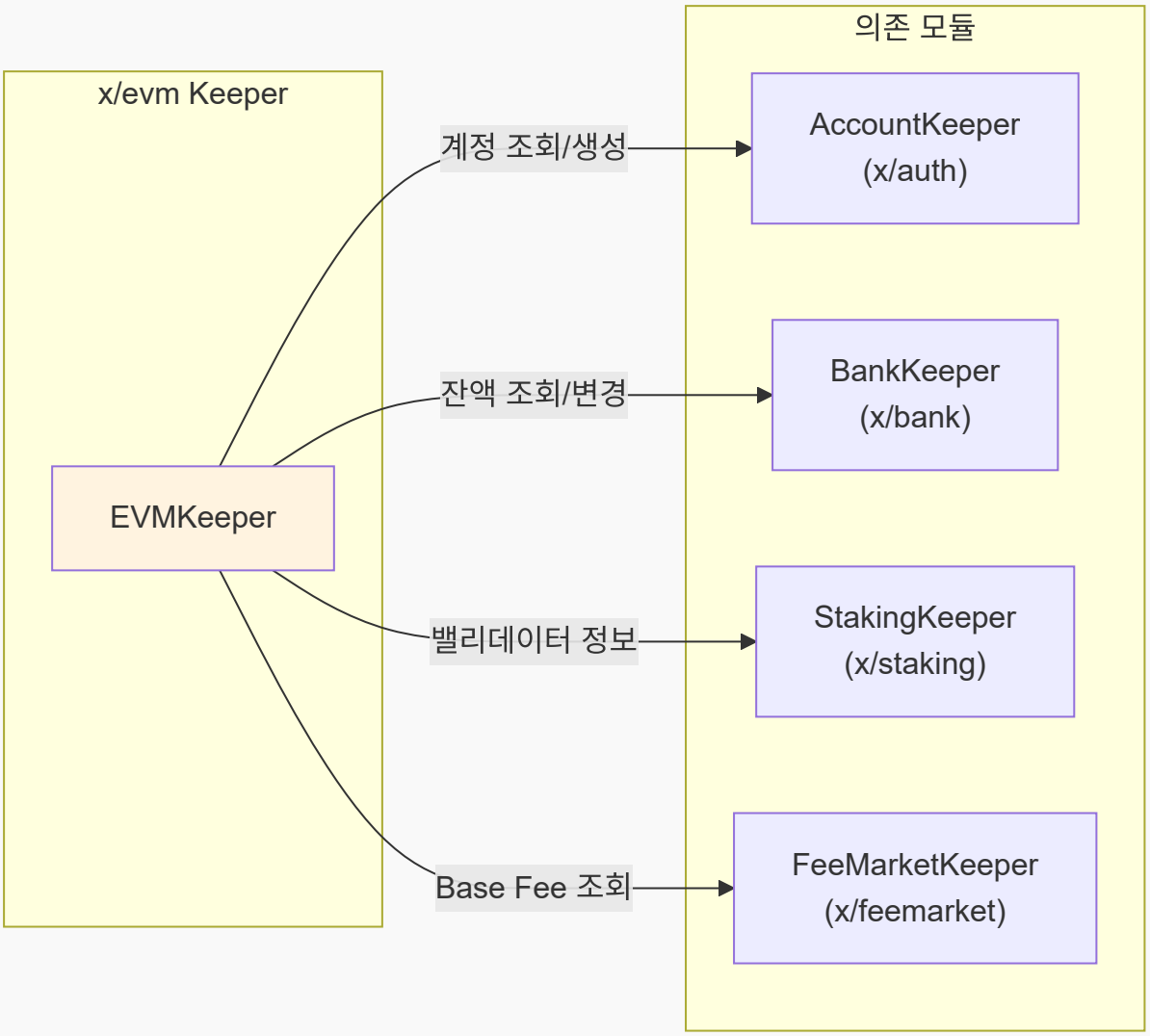
2.1 전체 구조

Cosmos EVM은 Cosmos SDK의 **모듈 시스템** 위에 구축됩니다. EVM 기능은 `x/evm`과 `x/feemarket` 두 개의 커스텀 모듈로 구현됩니다.


Cosmos EVM 전체 아키텍처
외부 인터페이스
ABCI
(CometBFT 연결)
gRPC/REST
(Cosmos 표준)
JSON-RPC
(eth_*, web3_*, net_*)
컨센서스 레이어
CometBFT
애플리케이션 레이어
AnteHandler
메시지 라우터
EVM 모듈
x/evm
(EVM 실행)
x/feemarket
(EIP-1559)
x/erc20
(토큰 변환)
코어 모듈
x/auth
(계정)
x/bank
(잔액)
x/staking
x/gov
상태 저장
MultiStore
(IAVL Tree)
TransientStore
(블록 내 임시)

2.2 모듈 간 의존성

Cosmos SDK의 **Keeper 패턴**에 따라, x/evm 모듈은 다른 모듈의 Keeper를 참조하여 기능을 수행합니다.




```
// EVMKeeper 의존성 (간략화)
type Keeper struct {
    storeKey        storetypes.StoreKey
    transientKey    storetypes.StoreKey

    // 외부 모듈 Keeper 참조
    accountKeeper   types.AccountKeeper
    bankKeeper      types.BankKeeper
    stakingKeeper   types.StakingKeeper
    feeMarketKeeper types.FeeMarketKeeper

    // EVM 관련
    tracer          string
    precompiles     map[common.Address]vm.PrecompiledContract
}
```

핵심 포인트: EVM의 계정 잔액은 실제로 `x/bank` 모듈에 저장되고, 계정 메타데이터는 `x/auth` 모듈에 저장됩니다. x/evm은 이들을 "래핑"하여 EVM 인터페이스로 노출합니다.

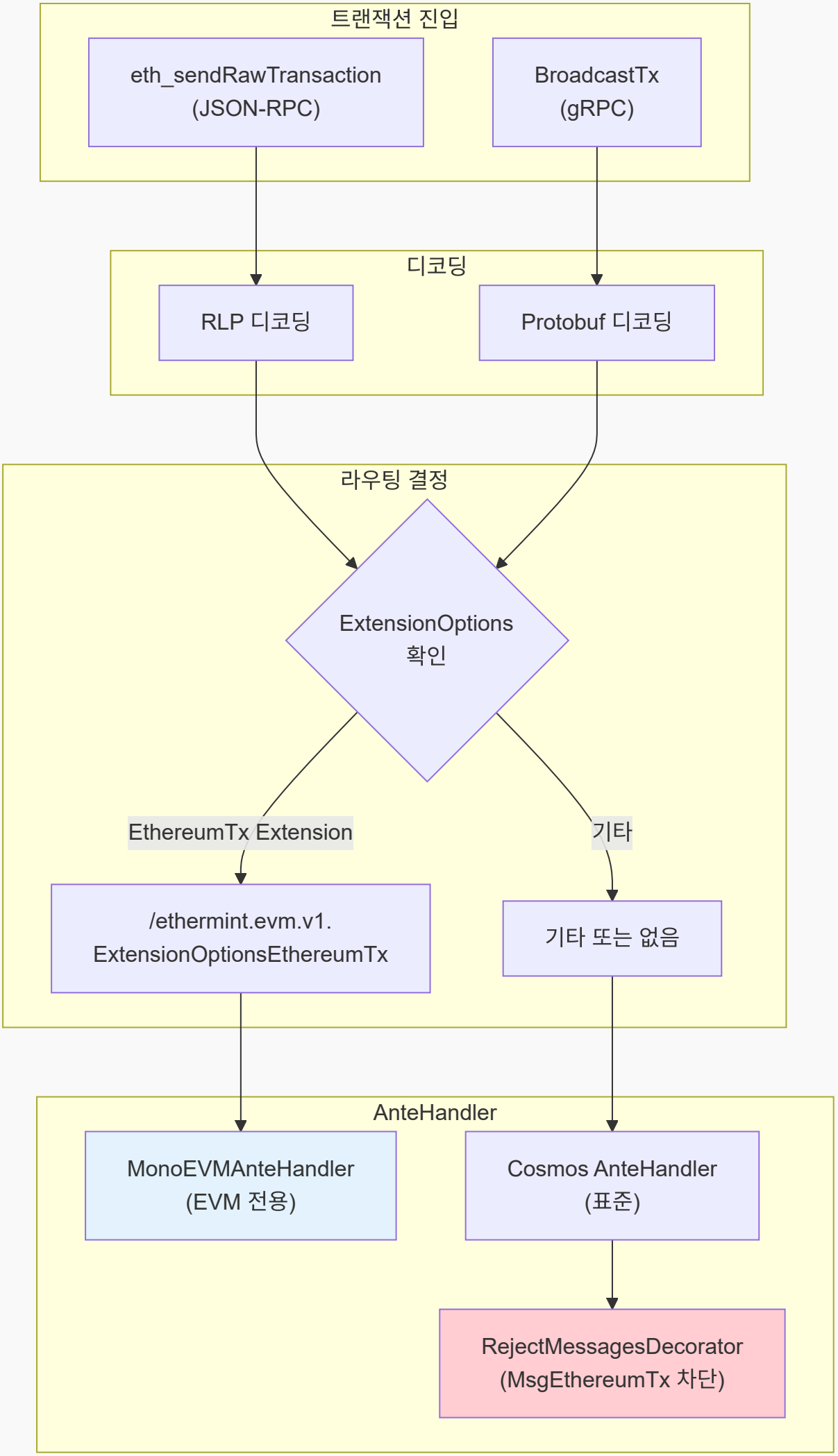
---

3. EVM 트랜잭션 처리 흐름

3.1 트랜잭션 유형 분기

Cosmos EVM은 **두 가지 트랜잭션 타입**을 처리합니다:

타입	소스	메시지 형식	AnteHandler
Ethereum Tx	MetaMask, ethers.js	`MsgEthereumTx`	`MonoEVMAnteHandler`
Cosmos Tx	Keplr, cosmjs	`MsgSend`, `MsgDelegate`...	Cosmos AnteHandler


트랜잭션 진입
eth_sendRawTransaction
(JSON-RPC)
BroadcastTx
(gRPC)
디코딩
RLP 디코딩
Protobuf 디코딩
라우팅 결정
ExtensionOptions
확인
EthereumTx Extension
기타
/ethermint.evm.v1.
ExtensionOptionsEthereumTx
기타 또는 없음
AnteHandler
MonoEVMAnteHandler
(EVM 전용)
Cosmos AnteHandler
(표준)
RejectMessagesDecorator
(MsgEthereumTx 차단)

> "A MsgEthereumTx with msg type of ethermint.evm.v1.ExtensionOptionsEthereumTx is first routed to its dedicated MonoEVMAnteHandler."
> — CertiK Research

3.2 MsgEthereumTx 구조

Ethereum 트랜잭션은 Cosmos SDK 메시지로 래핑됩니다:

```
// MsgEthereumTx 정의 (간략화)
message MsgEthereumTx {
    // Ethereum 서명자 주소 (검증용)
    bytes from = 1;

    // 완전한 Ethereum 트랜잭션 데이터 (RLP 인코딩)
    bytes raw = 2;
}
```

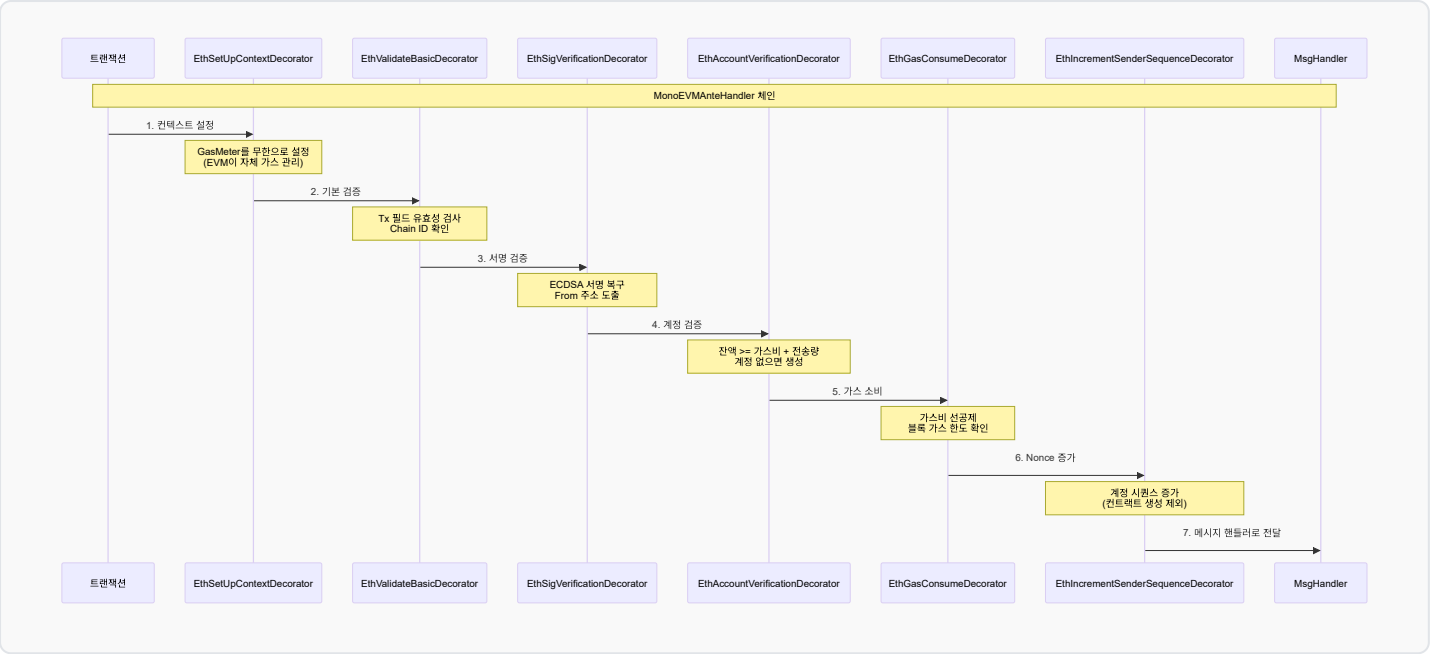
```
// MsgEthereumTx는 sdk.Msg와 sdk.Tx 인터페이스를 모두 구현
var (
    _ sdk.Msg = &MsgEthereumTx{}
    _ sdk.Tx  = &MsgEthereumTx{}
)
```

이중 인터페이스 구현의 의미:

- `sdk.Msg`: 메시지 핸들러로 전달되어 실행됨
- `sdk.Tx`: 단독으로 트랜잭션 역할 가능 (여러 메시지 포함 불필요)

3.3 AnteHandler 상세 흐름

EVM 트랜잭션의 AnteHandler는 여러 **Decorator** 체인으로 구성됩니다:

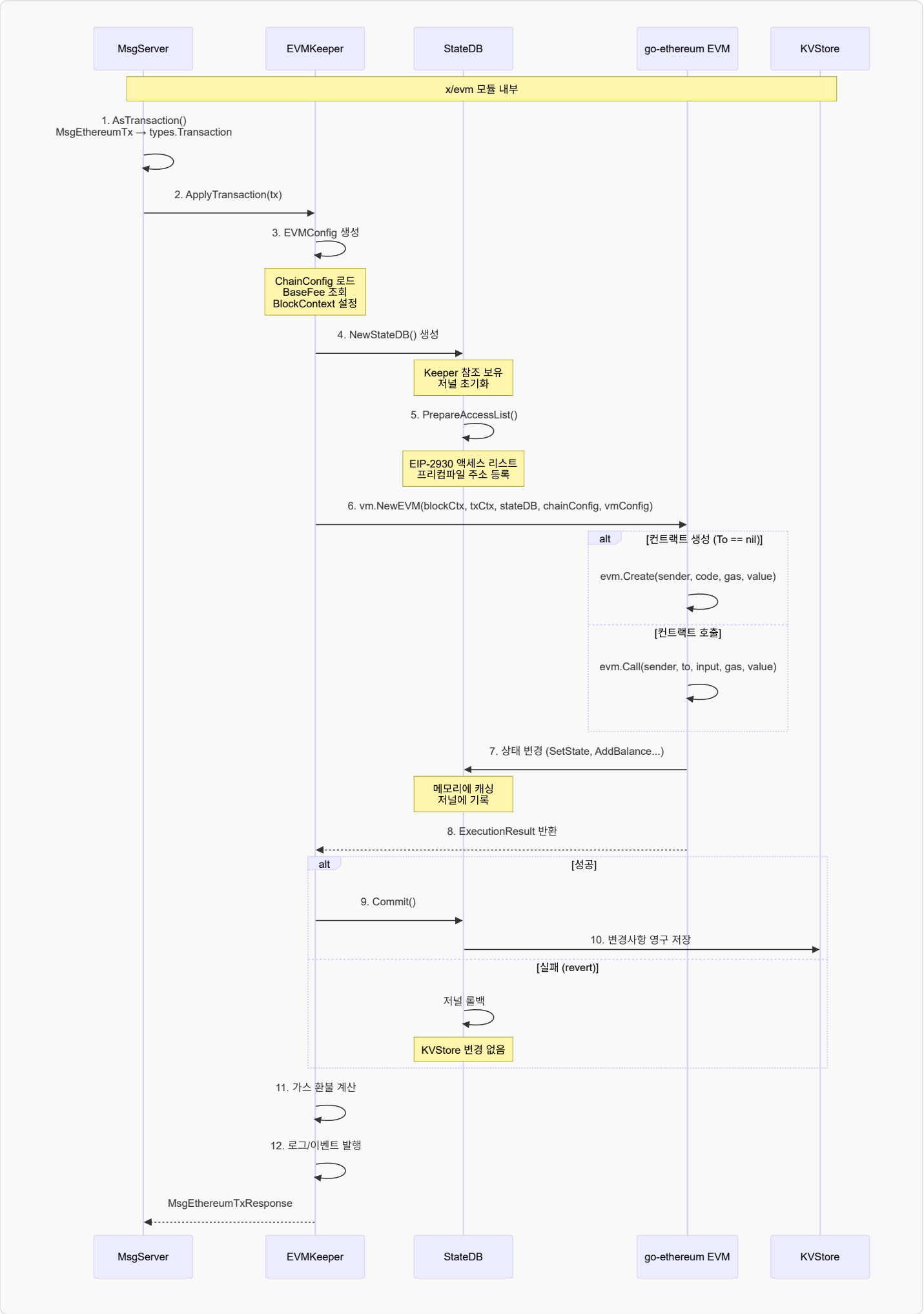


AnteHandler Decorator 역할 정리

Decorator	역할	실패 시
`EthSetUpContextDecorator`	GasMeter 무한 설정, 컨텍스트 초기화	-
`EthValidateBasicDecorator`	필드 검증, Chain ID 확인	트랜잭션 거부
`EthSigVerificationDecorator`	ECDSA 서명 검증, From 주소 도출	트랜잭션 거부
`EthAccountVerificationDecorator`	잔액 확인, 계정 자동 생성	트랜잭션 거부
`EthGasConsumeDecorator`	가스비 선공제, 블록 가스 한도 체크	트랜잭션 거부
`EthIncrementSenderSequenceDecorator`	Nonce(시퀀스) 증가	-

3.4 EVM 실행 흐름

AnteHandler 통과 후, 트랜잭션은 x/evm 모듈의 메시지 핸들러로 전달됩니다:


MsgServer
EVMKeeper
StateDB
go-ethereum EVM
KVStore
x/evm 모듈 내부
1. AsTransaction()
MsgEthereumTx → types.Transaction
2. ApplyTransaction(tx)
3. EVMConfig 생성
ChainConfig 로드
BaseFee 조회
BlockContext 설정
4. NewStateDB() 생성
Keeper 참조 보유
저널 초기화
5. PrepareAccessList()
EIP-2930 액세스 리스트
프리컴파일 주소 등록
6. vm.NewEVM(blockCtx, txCtx, stateDB, chainConfig, vmConfig)
alt
[컨트랙트 생성 (To == nil)]
evm.Create(sender, code, gas, value)
[컨트랙트 호출]
evm.Call(sender, to, input, gas, value)
7. 상태 변경 (SetState, AddBalance...)
메모리에 캐싱
저널에 기록
8. ExecutionResult 반환
alt
[성공]
9. Commit()
10. 변경사항 영구 저장
[실패 (revert)]
저널 롤백
KVStore 변경 없음
11. 가스 환불 계산
12. 로그/이벤트 발행
MsgEthereumTxResponse
MsgServer
EVMKeeper
StateDB
go-ethereum EVM
KVStore

핵심 코드 경로

```go
// 1. 메시지 서버 진입점
func (k *Keeper) EthereumTx(ctx context.Context, msg *types.MsgEthereumTx) (*types.MsgEthereumTxRespo
    // Cosmos 메시지 → Ethereum 트랜잭션 변환
    tx := msg.AsTransaction()

    // 트랜잭션 실행
    response, err := k.ApplyTransaction(ctx, tx)
    if err != nil {
        return nil, err
    }

    return response, nil
}

// 2. 트랜잭션 적용
func (k *Keeper) ApplyTransaction(ctx sdk.Context, tx *ethtypes.Transaction) (*types.MsgEthereumTxRes
    // EVM 설정 로드
    cfg, err := k.EVMConfig(ctx, tx)

    // 상태 전이 실행
    res, err := k.ApplyMessageWithConfig(ctx, msg, nil, true, cfg, txConfig)

    // 가스 환불
    k.RefundGas(ctx, msg, res.GasUsed, cfg.Params.EvmDenom)

    return res, nil
}

// 3. EVM 메시지 실행
func (k *Keeper) ApplyMessageWithConfig(ctx sdk.Context, msg core.Message, ...) (*types.MsgEthereumTx
    // StateDB 생성
    stateDB := statedb.New(ctx, k, txConfig)

    // go-ethereum EVM 인스턴스 생성
    evm := k.NewEVM(ctx, msg, cfg, tracer, stateDB)

    // 액세스 리스트 준비
    stateDB.PrepareAccessList(msg.From(), msg.To(), evm.ActivePrecompiles(rules), msg.AccessList())

    // EVM 실행!
    if contractCreation {
        ret, contractAddr, leftOverGas, vmErr = evm.Create(sender, msg.Data(), leftOverGas, msg.Value
    } else {
        ret, leftOverGas, vmErr = evm.Call(sender, *msg.To(), msg.Data(), leftOverGas, msg.Value())
    }

    // 성공 시 상태 커밋
    if vmErr == nil {
        if err := stateDB.Commit(); err != nil {
            return nil, err
        }
```

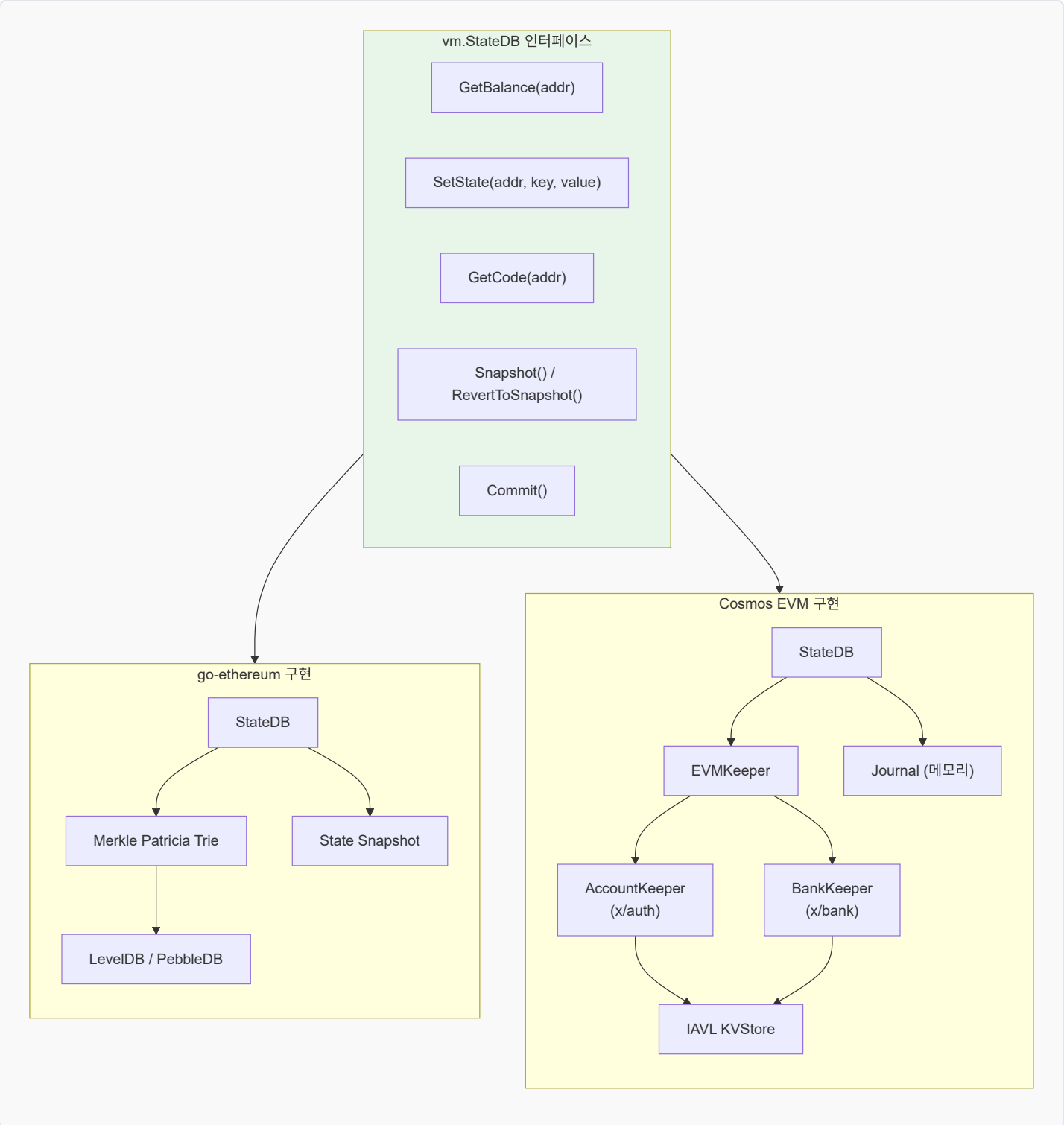
```
    }

    return &types.MsgEthereumTxResponse{...}, nil
}
```

---

4. StateDB: go-ethereum vs Cosmos EVM

4.1 아키텍처 비교

go-ethereum과 Cosmos EVM의 StateDB는 **동일한 인터페이스**(`vm.StateDB`)를 구현하지만, 내부 구현이 완전히 다릅니다.



4.2 상태 저장 방식 비교

구분	go-ethereum	Cosmos EVM
상태 트리	Merkle Patricia Trie (MPT)	IAVL Tree (via KVStore)
계정 저장	StateDB 내부 stateObject	x/auth AccountKeeper
잔액 저장	stateObject.Balance	x/bank BankKeeper
컨트랙트 코드	CodeHash → 별도 DB	EVM 모듈 KVStore
스토리지	storageTrie (계정별 MPT)	EVM 모듈 KVStore (prefix)
스냅샷/리버트	journal + revision	journal (동일 방식 차용)

4.3 StateDB 구현 상세

Cosmos EVM StateDB 구조

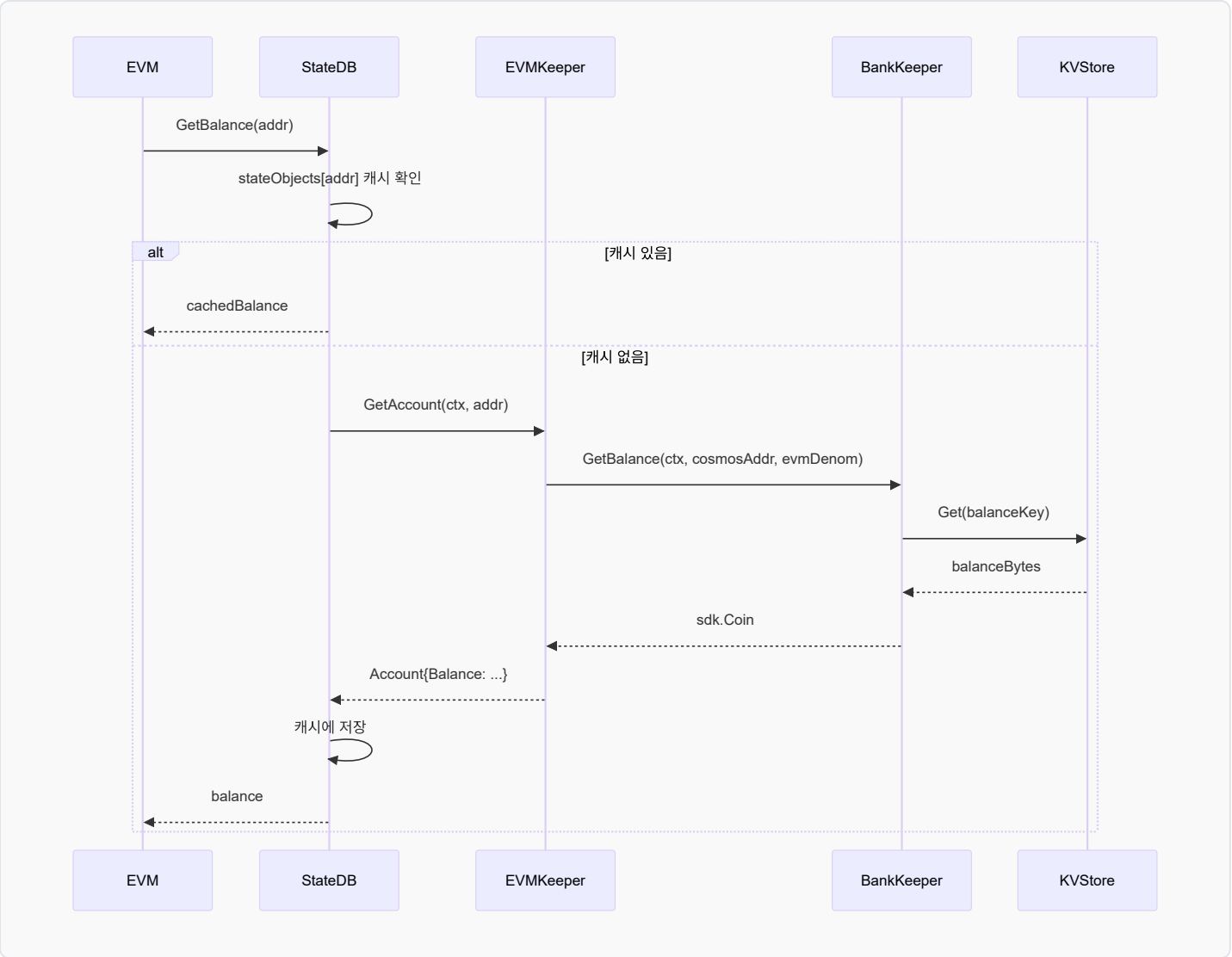
```
// Cosmos EVM StateDB (x/evm/statedb/statedb.go)
type StateDB struct {
    keeper Keeper           // EVMKeeper 참조
    ctx    sdk.Context      // Cosmos SDK 컨텍스트

    // 메모리 내 상태 캐시
    stateObjects map[common.Address]*stateObject

    // 저널링 (go-ethereum에서 차용)
    journal        *journal
    validRevisions []revision
    nextRevisionID int

    // EIP-2930 액세스 리스트
    accessList *accessList

    // 트랜잭션별 데이터
    txConfig TxConfig
    logs     []*ethtypes.Log
    refund   uint64
}

// revision: 스냅샷 식별자
type revision struct {
    id           int
    journalIndex int
}
```

stateObject 구조

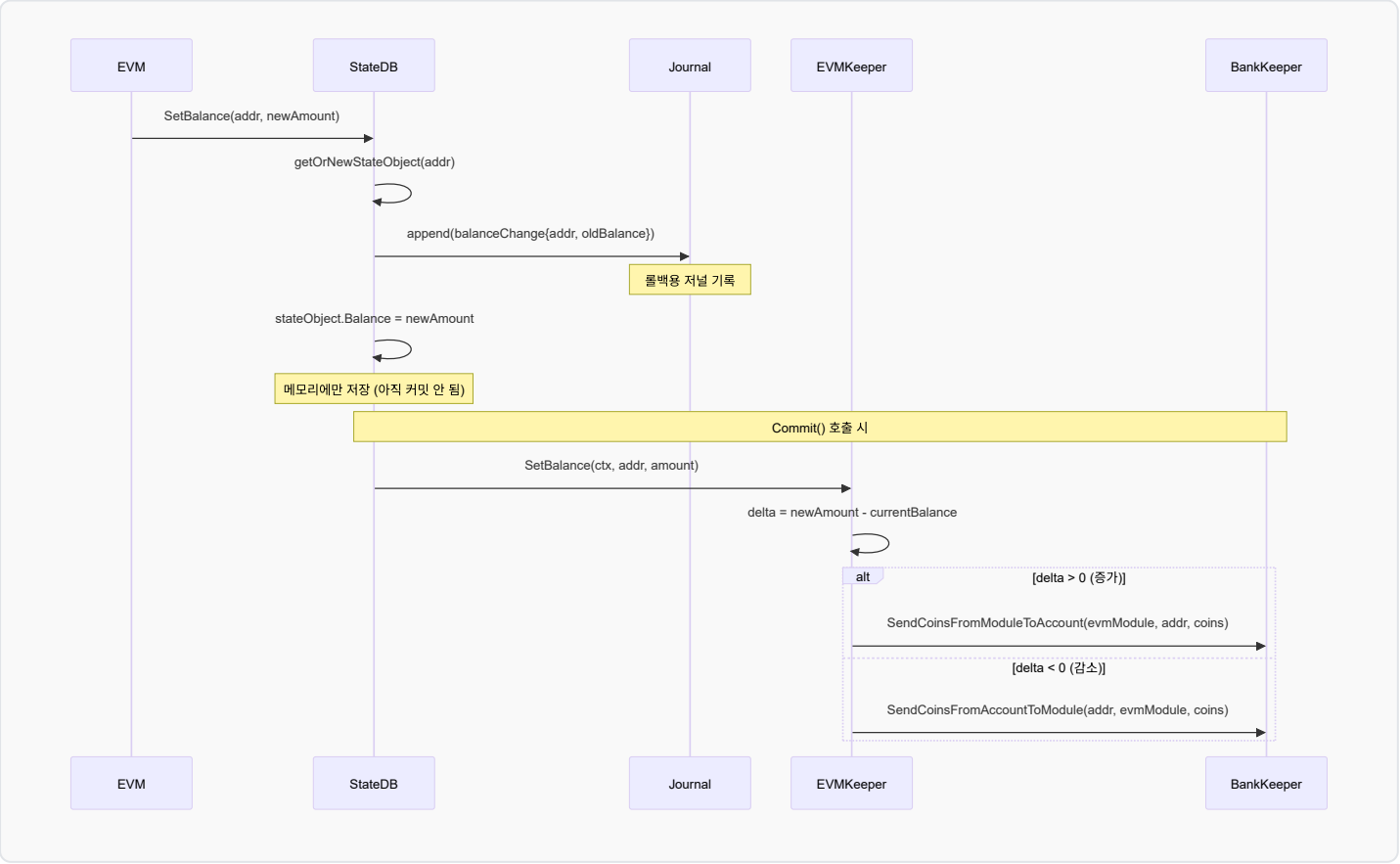
```
 // Cosmos EVM stateObject
type stateObject struct {
    address common.Address

    // 계정 데이터
    account Account  // Nonce, Balance, CodeHash

    // 스토리지 캐시
    originStorage Storage  // 원본 값
    dirtyStorage  Storage  // 변경된 값

    // 코드 캐시
    code     []byte
    codeDirty bool

    // 삭제 플래그
    suicided bool
}

// Account: Ethereum 계정 표현
type Account struct {
    Nonce    uint64
    Balance  *big.Int
    CodeHash []byte
}
```

4.4 상태 읽기/쓰기 흐름

잔액 조회 (GetBalance)



잔액 변경 (SetBalance)



핵심 차이점: go-ethereum은 잔액을 직접 stateObject에 저장하지만, Cosmos EVM은 **x/bank 모듈을 통해** 잔액을 관리합니다. 이로 인해:

1. **장점**: Cosmos SDK의 다른 모듈(staking, gov 등)과 잔액 공유
2. **단점**: 추가 추상화 레이어로 인한 오버헤드

4.5 스토리지 키-값 저장

go-ethereum 방식

```
Account Storage Trie (MPT)
├── keccak256(slot0) → value0
├── keccak256(slot1) → value1
└── ...
```

각 계정마다 별도의 Storage Trie가 있고, 이 루트 해시가 Account의 StorageRoot 필드에 저장됩니다.

Cosmos EVM 방식

```
KVStore (단일 네임스페이스)
├── prefix/address1/slot0 → value0
├── prefix/address1/slot1 → value1
├── prefix/address2/slot0 → value0
└── ...
```

```go
// 스토리지 키 생성
func AddressStoragePrefix(address common.Address) []byte {
    return append(KeyPrefixStorage, address.Bytes()...)
}

// 스토리지 읽기
func (k *Keeper) GetState(ctx sdk.Context, addr common.Address, key common.Hash) common.Hash {
    store := prefix.NewStore(ctx.KVStore(k.storeKey), types.AddressStoragePrefix(addr))
    value := store.Get(key.Bytes())
    if len(value) == 0 {
        return common.Hash{}
    }
    return common.BytesToHash(value)
}

// 스토리지 쓰기
func (k *Keeper) SetState(ctx sdk.Context, addr common.Address, key, value common.Hash) {
    store := prefix.NewStore(ctx.KVStore(k.storeKey), types.AddressStoragePrefix(addr))
    if value == (common.Hash{}) {
        store.Delete(key.Bytes())
    } else {
        store.Set(key.Bytes(), value.Bytes())
```

```
    }
}
```

4.6 저널링과 스냅샷

Cosmos EVM은 go-ethereum의 저널 시스템을 **거의 그대로 차용**했습니다:

```
 // journal.go - go-ethereum에서 차용
type journal struct {
    entries []journalEntry
}

type journalEntry interface {
    revert(*StateDB)
}

// 다양한 변경 유형
type (
    balanceChange struct {
        account common.Address
        prev    *big.Int
    }
    nonceChange struct {
        account common.Address
        prev    uint64
    }
    storageChange struct {
        account common.Address
        key     common.Hash
        prev    common.Hash
    }
    // ...
)

// 스냅샷 생성
func (s *StateDB) Snapshot() int {
    id := s.nextRevisionID
    s.nextRevisionID++
    s.validRevisions = append(s.validRevisions, revision{id, len(s.journal.entries)})
    return id
}

// 스냅샷으로 롤백
func (s *StateDB) RevertToSnapshot(id int) {
    // 해당 revision 찾기
    idx := sort.Search(len(s.validRevisions), func(i int) bool {
        return s.validRevisions[i].id >= id
    })

    snapshot := s.validRevisions[idx]
```

```
    // 저널 엔트리 역순 실행 (undo)
    for i := len(s.journal.entries) - 1; i >= snapshot.journalIndex; i-- {
        s.journal.entries[i].revert(s)
    }

    // 저널 truncate
    s.journal.entries = s.journal.entries[:snapshot.journalIndex]
}
```

---

5. Precompile 비교

5.1 Ethereum 표준 Precompile

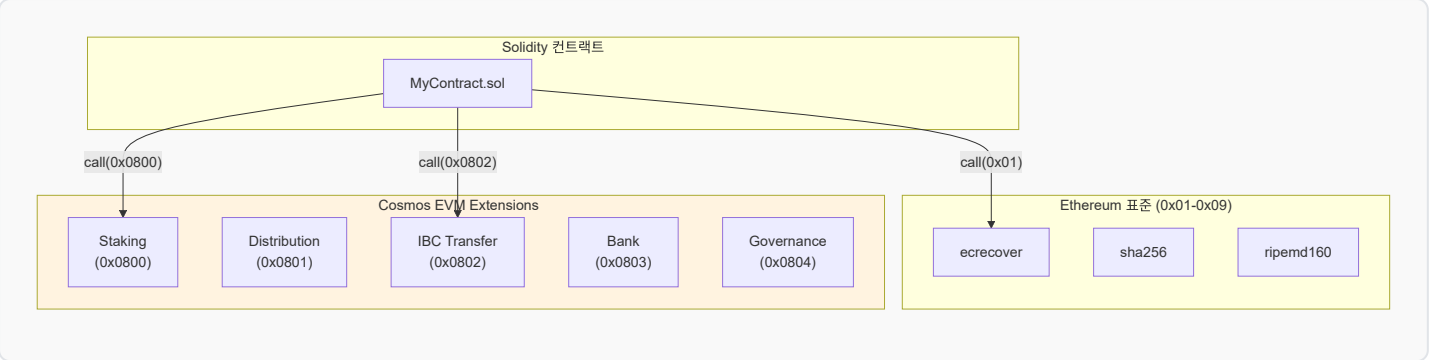
go-ethereum은 특정 주소에 **네이티브 구현**된 컨트랙트를 제공합니다:

주소	이름	기능
0x01	ecrecover	ECDSA 서명 복구
0x02	sha256	SHA-256 해시
0x03	ripemd160	RIPEMD-160 해시
0x04	identity	데이터 복사
0x05	modexp	모듈러 지수
0x06	bn256Add	BN256 곡선 덧셈
0x07	bn256ScalarMul	BN256 스칼라 곱
0x08	bn256Pairing	BN256 페어링
0x09	blake2f	BLAKE2 압축

```
 // go-ethereum precompile 인터페이스
type PrecompiledContract interface {
    RequiredGas(input []byte) uint64
    Run(input []byte) ([]byte, error)
}
```

5.2 Cosmos EVM Stateful Precompile (EVM Extensions)

Cosmos EVM은 **상태 변경이 가능한** Stateful Precompile을 추가합니다:



Stateful Precompile 인터페이스

```
// Cosmos EVM Stateful Precompile 인터페이스
type StatefulPrecompiledContract interface {
    PrecompiledContract

    // 상태 변경을 위한 추가 메서드
    RunStateful(
        ctx sdk.Context,
        evm *vm.EVM,
        contract *vm.Contract,
        stateDB vm.StateDB,
        input []byte,
    ) ([]byte, error)
}
```

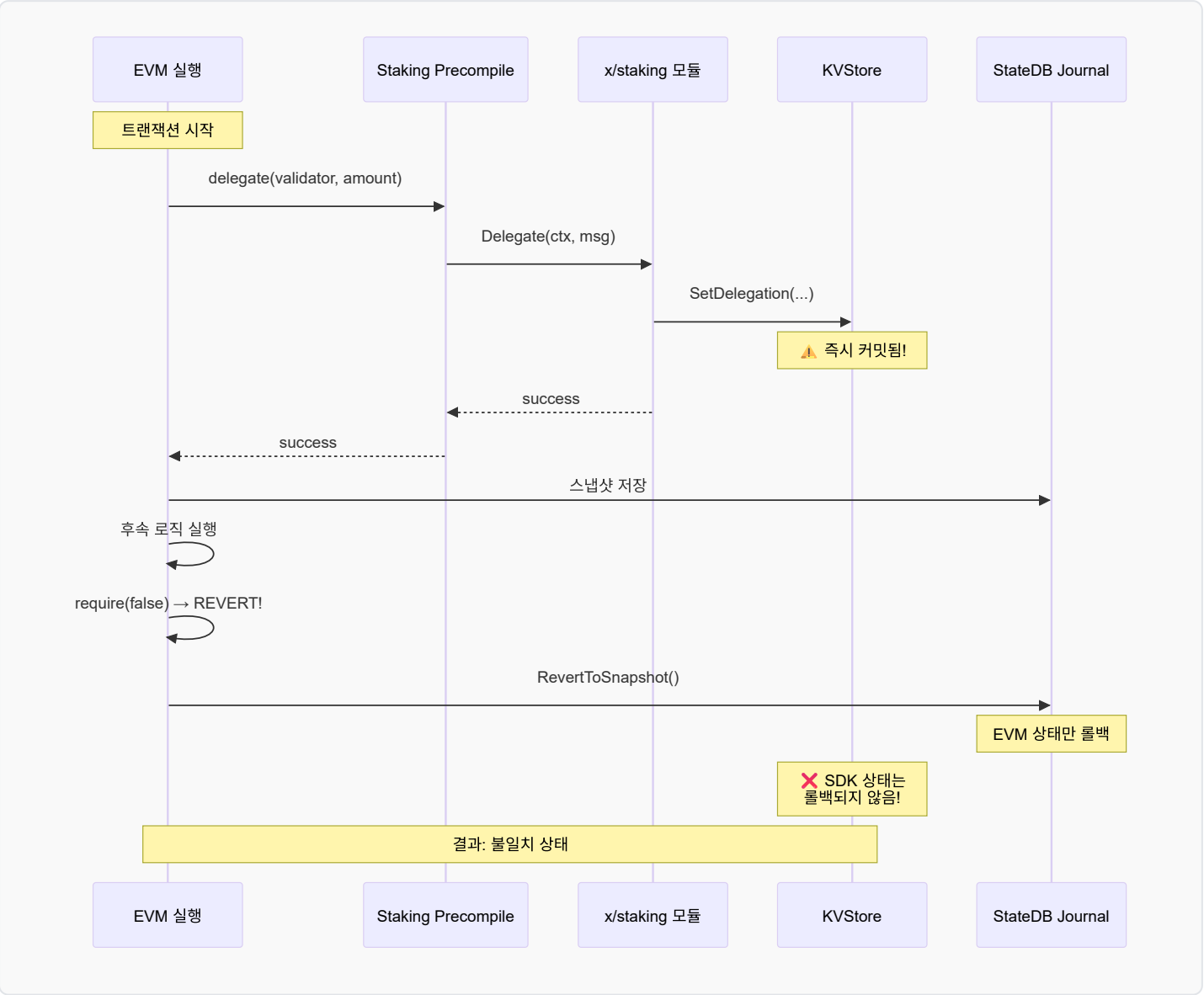
Staking Precompile 예시

```
// Solidity에서 Staking Precompile 호출
interface IStaking {
    function delegate(
        address delegatorAddress,
        string memory validatorAddress,
        uint256 amount
    ) external returns (bool success);

    function undelegate(
        address delegatorAddress,
        string memory validatorAddress,
        uint256 amount
    ) external returns (int64 completionTime);
}

contract StakingExample {
    IStaking constant STAKING = IStaking(0x0000000000000000000000000000000000000800);

    function stake(string memory validator, uint256 amount) external {
        STAKING.delegate(msg.sender, validator, amount);
```

```
    }
}
```

5.3 Stateful Precompile의 원자성 문제

핵심 문제: EVM의 revert와 Cosmos SDK의 상태 변경이 동기화되지 않습니다.



> "If we were to call a module's logic through a stateful precompile and NOT use the journal semantics, even though the EVM transition reverted, the KVStore changes would persist."
> — Ethermint GitHub Issue #1363

해결책: RunAtomic 패턴

```
// Evmos의 해결책: 원자적 실행 래퍼
func (p *Precompile) RunAtomic(
    ctx sdk.Context,
    evm *vm.EVM,
    contract *vm.Contract,
    stateDB vm.StateDB,
    input []byte,
```

```
) (bz []byte, err error) {
    // 1. 캐시 컨텍스트 생성
    cacheCtx, commit := ctx.CacheContext()

    // 2. StateDB 스냅샷 저장
    snapshot := stateDB.Snapshot()

    // 3. 실제 로직 실행
    bz, err = p.run(cacheCtx, evm, contract, stateDB, input)

    if err != nil {
        // 4a. 실패 시: StateDB 롤백 + 캐시 컨텍스트 버림
        stateDB.RevertToSnapshot(snapshot)
        return nil, err
    }

    // 4b. 성공 시: 캐시 컨텍스트 커밋
    commit()
    return bz, nil
}
```

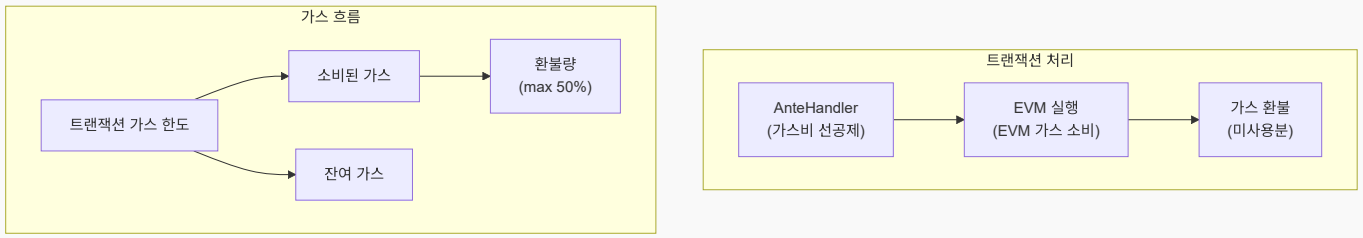
---

6. 가스 모델 비교

6.1 이중 가스 시스템

Cosmos EVM에서는 **두 개의 가스 시스템**이 공존합니다:

시스템	용도	특징
EVM Gas	EVM opcode 실행 비용	환불 메커니즘 있음
Cosmos Gas	SDK 모듈 연산 비용	환불 없음



6.2 가스 계량 분리

```
 // AnteHandler에서 Cosmos GasMeter를 무한으로 설정
func (d EthSetUpContextDecorator) AnteHandle(ctx sdk.Context, tx sdk.Tx, simulate bool, next sdk.Ante
    // Cosmos SDK 가스 계량 비활성화
    // EVM이 자체적으로 가스를 관리
```

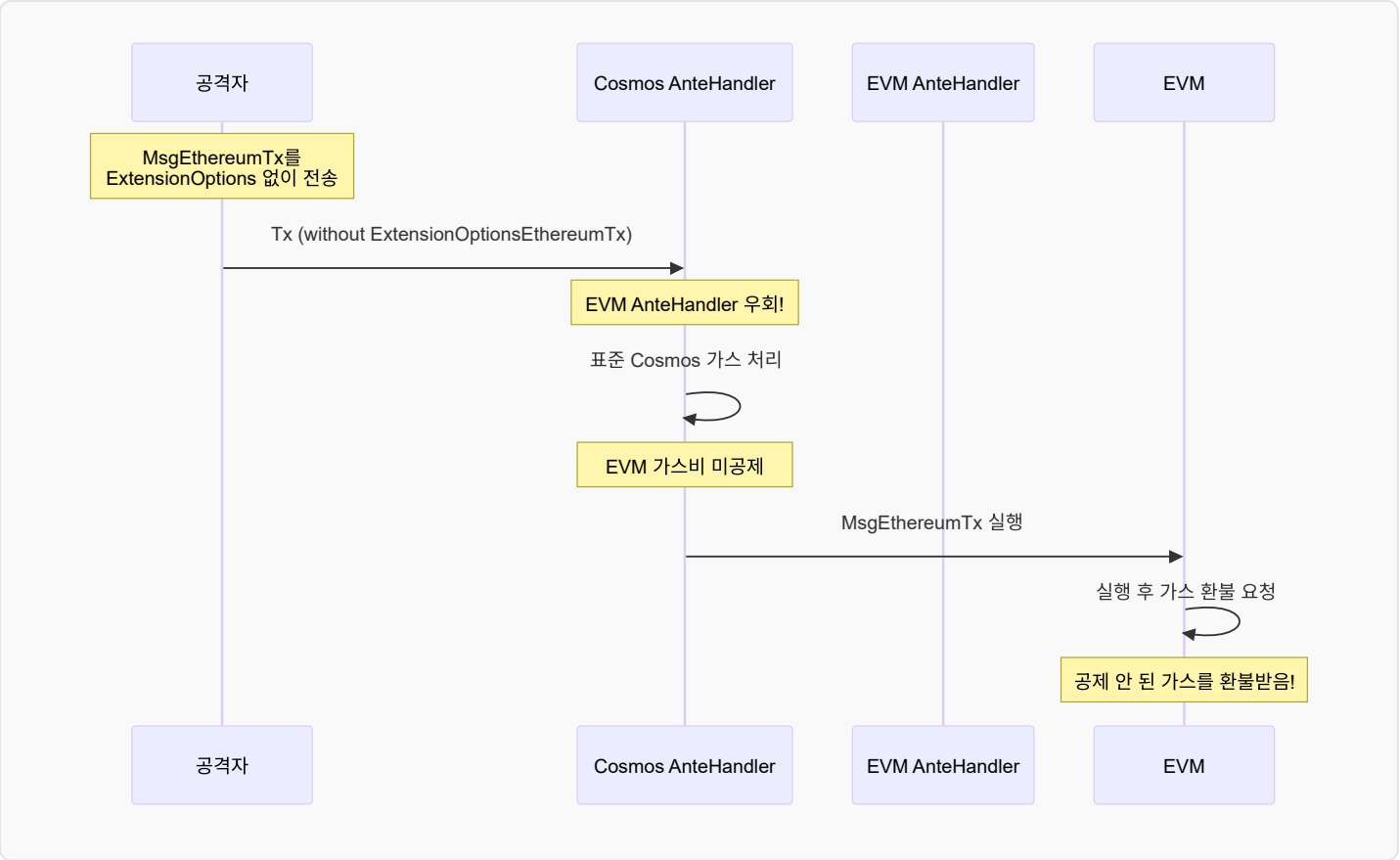
```
    newCtx := ctx.WithGasMeter(storetypes.NewInfiniteGasMeter())

    return next(newCtx, tx, simulate)
}
```

```
 // EVM 실행 시 가스 관리
func (k *Keeper) ApplyMessageWithConfig(...) {
    // EVM이 자체 가스 계량
    leftOverGas := msg.Gas()

    // 실행
    if contractCreation {
        ret, _, leftOverGas, vmErr = evm.Create(...)
    } else {
        ret, leftOverGas, vmErr = evm.Call(...)
    }

    // 가스 환불 계산 (EIP-3529 적용)
    gasUsed := msg.Gas() - leftOverGas
    refundQuotient := params.RefundQuotient  // 5 (London 이후)
    gasRefund := min(stateDB.GetRefund(), gasUsed / refundQuotient)

    // 환불 실행
    k.RefundGas(ctx, msg, gasUsed - gasRefund, cfg.Params.EvmDenom)
}
```

6.3 가스 관련 보안 취약점

> "Multiple critical vulnerabilities have been disclosed in EVMOS-style chains, all involving transaction fee manipulation."
> — CertiK Research

문제 시나리오:



해결책: `RejectMessagesDecorator`

```
// Cosmos AnteHandler에서 MsgEthereumTx 차단
func (rmd RejectMessagesDecorator) AnteHandle(ctx sdk.Context, tx sdk.Tx, simulate bool, next sdk.Ant
    for _, msg := range tx.GetMsgs() {
        if _, ok := msg.(*evmtypes.MsgEthereumTx); ok {
            return ctx, errorsmod.Wrapf(
                errortypes.ErrInvalidType,
                "MsgEthereumTx needs ExtensionOptionsEthereumTx",
            )
        }
    }
    return next(ctx, tx, simulate)
}
```

---

7. 종합 비교 요약

7.1 아키텍처 비교표

구성 요소	go-ethereum	Cosmos EVM
상태 트리	Merkle Patricia Trie	IAVL Tree (via KVStore)
계정 관리	내장 (stateObject)	x/auth 모듈 위임

구성 요소	go-ethereum	Cosmos EVM
잔액 관리	내장	x/bank 모듈 위임
스토리지	계정별 StorageTrie	단일 KVStore (prefix)
저널링	내장 journal	동일 방식 차용
Precompile	Stateless (0x01-0x09)	Stateless + Stateful Extensions
가스 모델	단일 (EVM Gas)	이중 (EVM + Cosmos)
합의	PoW → PoS (Beacon)	CometBFT (BFT)
트랜잭션 형식	RLP 인코딩	MsgEthereumTx (Protobuf 래핑)

7.2 트레이드오프

7.3 핵심 설계 결정 요약

1. **geth를 라이브러리로 사용**: EVM 바이트코드 인터프리터는 go-ethereum 그대로 사용하되, StateDB만 재구현

2. **이중 트랜잭션 시스템**: Ethereum TX와 Cosmos TX 모두 지원, ExtensionOptions로 라우팅

3. **모듈 간 책임 분리**:

   - x/auth: 계정 메타데이터
   - x/bank: 잔액
   - x/evm: EVM 실행, 컨트랙트 코드, 스토리지

4. **저널 기반 롤백**: go-ethereum의 journal 시스템을 차용하여 스냅샷/리버트 구현

5. **Stateful Precompile**: Cosmos SDK 기능을 EVM에 노출, 원자성은 CacheContext로 보장

---

8. 결론

Cosmos EVM은 "두 세계의 장점"을 결합하려는 야심찬 프로젝트입니다. go-ethereum의 검증된 EVM 구현을 재사용하면서, Cosmos SDK의 모듈성과 IBC 인터오퍼러빌리티를 제공합니다.

그러나 이 통합은 공짜가 아닙니다:

1. **StateDB 재구현**은 복잡하고, 미묘한 버그의 원인이 됩니다
2. **두 가스 시스템**의 공존은 보안 취약점을 만들 수 있습니다
3. **Stateful Precompile**은 원자성 보장을 위해 추가적인 래핑이 필요합니다

> "The current design has been a major barrier towards getting Berachain to a level of developer UX that is familiar to EVM folk, while also ensuring we can leverage the SDK and IBC to its full potential."
> — Berachain (Ethermint Issue #1363)

이러한 도전들이 바로 Berachain이 BeaconKit으로, Omni가 Octane으로 **Separated Execution** 방식을 선택한 이유입니다. 하지만 Cosmos SDK 모듈과의 깊은 통합이 필요한 프로젝트에게 Cosmos EVM은 여전히 유효한 선택입니다.

참고 자료

코드 저장소

- cosmos/evm: https://github.com/cosmos/evm
- evmos/ethermint: https://github.com/evmos/ethermint
- ethereum/go-ethereum: https://github.com/ethereum/go-ethereum

공식 문서

- Cosmos EVM Documentation: https://evm.cosmos.network
- go-ethereum Documentation: https://geth.ethereum.org/docs

보안 연구

- CertiK: EVM-Cosmos Convergence Research (Part 1, 2, 3)
- Jump Crypto: Bypassing Ethermint Ante Handlers
- Ethermint GitHub Issues: #1363 (Stateful Precompile UX), #710 (StateDB Performance)